

Server API Surface Extensions for Multi-Account Helix OTA — To-Be Design

Revision: 1 Last modified: 2026-07-10T11:18:54Z

Scope. This doc designs the HTTP **API surface** the multi-account feature adds to the Helix OTA control plane (`server/`, Go/Gin modular monolith): the super-admin admin API (`accounts/users/memberships/roles`), how the existing OTA routes become account-scoped, the post-sign-in account-selection API, the credentials API (`project-CLI` API keys + device provisioning tokens), the missing endpoints the as-is audit flagged, and the versioning / backward-compat / error / pagination contracts. It is the route-shape companion to the data-model keystone.

Authority to defer to (cited, never contradicted). Entity shapes (`Account / User / Membership / Role / api_keys` columns / token claim) are owned by `20_target_multitenancy_data_model.md` (**SSOT**) — this doc *consumes* those shapes and the Repository methods it declares, and invents no conflicting shape. The permission-model *shape* (RBAC vs ABAC vs role+scope — OQ-2) is owned by `21_authz_rbac_superadmin.md`; the token-claim field names + the super-admin claim encoding are `21_*`'s to finalize — this doc fixes only route shapes + authZ gates + the claim-derivation *direction*. The project-CLI auth model (Option A/B), the object-storage dependency, and the per-account signing-key registry are owned by `24_project_side_cli.md`; the device enrollment / provisioning model by `25_device_side_update_client.md`; cross-tenant isolation proofs + the anti-enumeration rule by `26_security_threat_model.md`; UI consumption by `23_ui_ux_all_surfaces.md`.

Reading order. Read `00_INDEX.md`, `10_existing_auth_and_project_model.md`, `11_existing_upload_and_device_update.md`, and `20_*` FIRST. Every “as-is” claim about an existing route carries a first-hand

file:line; every “to-be” element is a proposal, presented as a recommendation-with-tradeoffs for operator decision — never a silent decision (§11.4.6 / §11.4.66).

1. Super-admin admin API — accounts / users / memberships / roles

1.1 As-is: no admin surface, no super-admin gate

Today there is **no accounts/users/memberships surface at all** — the route table (server/internal/api/server.go:183-251) exposes only auth, devices, artifacts, deltas, releases, deployments, rollout, client, telemetry, groups, audit, and projects. Projects are the closest precedent: POST/GET/GET/PATCH/DELETE /projects (server.go:246-250), gated by the **global** requireRole (middleware.go:79-97) plus, on the single-project routes, the per-project ACL requireProjectAccess (handlers_project.go:37-74). There is **no persisted User entity** — identity is the token Subject and an env-seeded in-memory StaticUserDirectory (users.go:8-48; bootstrap main.go:96-104) — and **no is_super_admin concept**; a global RBAC admin merely bypasses the project ACL (handlers_project.go:43-60). 20_* §1.2/§6 introduces the persisted User + the global users.is_super_admin boolean this admin API gates on.

1.2 New route namespace + the super-admin gate

The admin API is a **new, additive namespace** under the existing /api/v1 group (server.go:189-190 — auth := v1.Group("") with authMiddleware() + auditMiddleware()). It is gated by a **new requireSuperAdmin middleware** (sibling of requireRole, middleware.go:79-97) that admits only a principal whose resolved identity carries is_super_admin (the claim/flag shape is 21_*'s; 20_* §6 is the data-layer source). requireSuperAdmin is strictly stronger than the global admin role — a normal account admin may administer *within* their account (§2) but never create accounts, provision users, or read another account’s data.

Method + path (new)	Handler (new)	AuthZ	Repository me (20_* §3.1)
POST /admin/accounts	handleCreateAccount	requireSuperAdmin	CreateAccountW (atomic, 20_* §3.1)

Method + path (new)	Handler (new)	AuthZ	Repository me (20_* §3.1)
GET /admin/accounts	handleListAccounts	requireSuperAdmin	ListAccounts
GET /admin/ accounts/:accountId	handleGetAccount	requireSuperAdmin	GetAccount
PATCH /admin/ accounts/:accountId	handleUpdateAccount	requireSuperAdmin	UpdateAccount (slug/status)
DELETE /admin/ accounts/:accountId	handleDeleteAccount	requireSuperAdmin	DeleteAccount (per 20_* §4 FKS)
POST /admin/users	handleCreateUser	requireSuperAdmin	CreateUser
GET /admin/users	handleListUsers	requireSuperAdmin	ListUsers
GET /admin/ users/:userId	handleGetUser	requireSuperAdmin	GetUser
PATCH /admin/ users/:userId	handleUpdateUser	requireSuperAdmin	UpdateUser (email/active/is_superuser)
DELETE /admin/ users/:userId	handleDeleteUser	requireSuperAdmin	DeleteUser
PUT /admin/ accounts/:accountId/ members/:userId	handleSetMembership	requireSuperAdmin	SetAccountMembership (grant/change revoke)
GET /admin/ accounts/:accountId/ members	handleListAccountMembers	requireSuperAdmin	ListAccountMembers
DELETE /admin/ accounts/:accountId/ members/:userId	handleRemoveMembership	requireSuperAdmin	RemoveAccountMembership

† **Trust-boundary caveat on is_superuser.** Per 20_* §6 the super-admin flag is maximum-blast-radius and is settable **only via config/env bootstrap, never a request** — the same rule the codebase already enforces for the token secret (config.go:180-184), the TLS-proxy trust flag (config.go:88-110), and the artifact-verify key (resolvePublicKey, handlers_artifact.go:274-283). **Recommendation:** PATCH /admin/users/:userId accepts profile/active edits but MUST reject any attempt to set is_superuser=true over the wire (a 403, or silently ignore the field) — promotion to super-admin stays a bootstrap-only operation. *Tradeoff:* an operator who wants runtime super-admin promotion via the API accepts widening that trust boundary; the safe default is

config-only, and 26_* owns the final rule. **This admin API assumes at least one super-admin already exists** (env bootstrap, main.go:96-104 extended per 20_* §6) — it never mints the *first* one.

1.3 Role / permission assignment — route shape only, model deferred

Per-account **role assignment** rides on PUT /admin/accounts/:accountId/members/:userId above (the membership row carries the role — 20_* §1.3). The richer **per-account role/permission catalog** (“ultimate flexibility”, OQ-2) — if 21_* chooses the reserved roles/permissions tables (20_* §1.4) — adds an additive sub-namespace whose shape this doc sketches but does **not** decide:

Method + path (conditional on 21_* choosing table- based RBAC)	AuthZ	Notes
POST/GET/PATCH/ DELETE /admin/ accounts/:accountId/ roles[...]	requireSuperAdmin (+ account admin if 21_* delegates)	tenant-scoped role catalog (roles(account_id,name), 20_* §1.4)
PUT /admin/ accounts/:accountId/ roles/:roleId/ permissions	as above	attach permissions to a role

Recommendation: ship only membership-role assignment (fixed viewer<operator<admin enum, reusing the ProjectRole rank tooling handlers_project.go:78-92) in the first milestone, and gate the role/permission-catalog routes behind 21_*’s OQ-2 decision so they are a pure additive migration. *Tradeoff:* the fixed enum caps flexibility at three roles now; the catalog routes deliver the mandated max-flexibility later at the cost of a role-resolution join per authZ check (20_* §1.4). No silent choice — 21_* owns it.

2. Account-scoping the existing OTA routes

This is the load-bearing decision: **where does tenancy live in the request, and how is it derived.** The as-is state (established first-hand): the token Claims{sub,roles,iat,exp} carry **no tenant dimension**

(token.go:31-36), and every OTA route is gated **only** by the global requireRole — devices (server.go:192-195), artifacts (:197-198), deltas (:202-203), releases (:205-207), deployments (:209-211), rollout (:215-217), recall (:220-221), client update/telemetry (:223-224), groups (:232-239), audit (:242). None consults account or project, because none carries one (10_* §2, §4).

2.0 The anchor decision: token-claim tenancy (primary) + path tenancy (admin only)

Two canonical ways to put a tenant in a REST request (Microsoft Azure Architecture Center; apyflux; the multi-tenant-SaaS JWT-claims guide — §Sources):

- **Path-based** — /api/v1/accounts/{accountId}/artifacts/upload. The tenant is explicit in the URL. *Pro*: self-documenting, trivially routable, easy to log/cache per tenant. *Con*: the tenant id is **client-supplied**, so it MUST still be authorized against the caller's identity on every request — “even though the request includes a domain name or other tenant identifier, it doesn't mean you should automatically grant access” (Azure, *Request validation*, assume-breach). It also churns **every** OTA path + every client call site (SPA, CLI, device agent).
- **Token-claim** — the account (and, where relevant, project) is a **top-level immutable claim inside the signed token**; the server reads it from that one place, and **the paths stay unchanged**. “If tenant_id is a top-level, immutable claim inside the signed payload, and every layer reads it from that one place, the boundary is enforced by the same cryptography that protects identity ... if tenant scope is reconstructed from request metadata after verification, the signature protects nothing useful” (JWT-claims guide, §Sources).

Recommendation (hybrid):

1. **Primary hot path — account is derived from the signed token claim, NEVER from the path or a header.** The existing OTA routes keep their current paths (server.go:192-242) and gain a **scoping middleware** (§2.1) that reads account_id (+ project_id where present) from the verified Claims. This preserves the trust boundary 20_* §6 / 24_* §1 / 25_* §1.2 fix (tenancy is server-minted + server-verified, the client can never assert its own account), it is the smallest path diff (no URL churn for device/CLI/SPA callers), and it is exactly the model 23_* §1.3, 24_* §1.3 Option B, and 25_* §1.2 already assume.

2. **Admin + super-admin cross-account operations — path-based** (/admin/accounts/:id/..., §1), because there the caller legitimately acts across many accounts and **MUST** name the target account explicitly; that path segment is authorized against `is_super_admin` (§1.2), so it is a named target, never a self-asserted scope.

Tradeoff of the token-claim primary: switching the active account requires a token re-exchange (§3) rather than just changing a URL — accepted, because it is the same “one token per active tenant session, switch context explicitly” pattern the JWT-claims guide recommends and 23_*/24_* already assume. An operator who prefers explicit per-request account paths can layer the path form on top later (they compose); this doc does not silently foreclose it.

2.1 The scoping middleware (`requireAccountAccess`)

A new middleware runs **after** `authMiddleware` (`server.go:190`) on every account-scoped route, mirroring the shape of `requireProjectAccess` (`handlers_project.go:37-74`):

1. Read `account_id` (+ optional `project_id`) from the verified Claims (never a header or body field — the JWT-claims / Azure assume-breach rule).
2. Resolve the caller’s membership via `GetAccountMembership(userID, accountID)` (20_* §3.1) and deny below the route’s `minRole` (reuse the `viewer<operator<admin` rank compare, `handlers_project.go:78-92`).
3. **Super-admin bypass** mirrors the *exact precedent already in the code*: a global admin/super-admin bypasses the per-project ACL and sees all projects (`handlers_project.go:43-60`, `list-all :162-199`); the account layer generalizes this — `is_super_admin` skips the `account_id` membership filter (20_* §6).
4. Stash the resolved (`account_id`, `project_id`) in the Gin context so handlers + the store layer scope reads/writes. The Repository accessors gain the scope per 20_* §3.2 (explicit `accountID` param on single-entity methods + `AccountID/ProjectID` fields on the existing `*Filter` structs `store.go:259-285` for list paths), with pgx RLS as the independent second layer (20_* §0).

Per-route effect (route paths unchanged; the gate + the derived scope are the change):

Existing route (<code>server.go</code>)	Added gate	Scope source	Newly-
	<code>requireAccountAccess(operator)</code>		

Existing route (server.go)	Added gate	Scope source	Newly-
POST /artifacts/ upload (:197)		claim account_id + --project validated against claim's authorized set	artifact account
POST /releases (:205)	requireAccountAccess(operator)	claim	LatestR per-(ac (store.
POST /deployments (:209)	requireAccountAccess(operator)	claim	active-c (account (store.
GET /devices / / releases / / deployments / / groups / /audit (:193,206,210,233,242)	requireAccountAccess(viewer\ admin)	claim → *Filter.AccountID	list resu account
GET /client/update + POST /client/ telemetry (:223-224, RoleDevice)	claim account_id on the device token	server-minted at register (25_* §1.2)	ActiveD account ever se
POST /devices/ register (:192)	requireAccountAccess(operator)	claim	mintDev (handle account token (
/projects CRUD (:246-250)	requireAccountAccess	claim	project unique

The **project** dimension stays the caller-supplied --project (query/body) but is **validated against the token's authorized project set** and rejected if out of scope (requireProjectAccess pattern, handlers_project.go:37-74; 24_* §3 step 2) — project is a narrowing *within* the claim's account, never a widening of it.

3. Account-selection API (post-sign-in)

3.1 As-is: login returns a role-only token, no account step

POST /api/v1/auth/login → handleLogin(handlers_auth.go:77-99) authenticates via the injected UserDirectory and calls issueTokenPair (:123-137), which mints an access token (signer.Mint, roles only) + an opaque single-use-rotating refresh token (refreshStore, :28-72; 30-day default :19) and returns

TokenResponse{AccessToken, TokenType, ExpiresIn, RefreshToken, Roles} (:130-136). POST /auth/refresh → handleRefresh (:102-119) rotates the refresh token. **There is no account dimension anywhere in this flow.**

3.2 New: list-my-accounts + set-active-account (token exchange)

The mandate wants “account selection after sign-in” (00_INDEX.md §1 item 5). Two new public/authenticated routes, additive beside the existing auth pair (server.go:184-185):

Method + path (new)	AuthZ	Purpose	Backing
GET /auth/accounts (a.k.a. “list my accounts”)	authMiddleware (identity-scoped token)	after sign-in, list the caller’s memberships → the picker (23_* §2.1)	ListUserAccounts(userID) (20_* §3.1) → [{account_id, name, slug, role, is_owner}]
POST /auth/select-account (token exchange)	authMiddleware	exchange the identity token for a scoped access token carrying the account_id (+ project) claim	validate GetAccountMembership(userID, accountID); re-mint via an issueTokenPair extended with the tenant claim (handlers_auth.go:123; claim shape = 21_*)

Design (recommendation): identity-scoped refresh token + per-account short-lived access token minted on selection/switch. This is the WorkOS/JWT-claims “one token per active tenant session, switch

context explicitly” pattern (§Sources; 23_* §1.3): the long-lived refresh credential stays identity-scoped, and the account claim is **derived on demand** and never embedded in the long-lived credential — so a compromised/rotated session cannot silently act in the wrong account. On account **switch**, the client calls POST /auth/select-account again for the new account and invalidates account-scoped caches (23_* §2.3).

- **Single membership** → the server MAY auto-issue the scoped token at login (skip the picker — Clerk pattern, 23_* §2.1). **Zero memberships** → GET /auth/accounts returns an empty list and the client shows the “no account access” dead-end (never a silent blank). **Super-admin** → GET /auth/accounts returns the System scope; an explicit “act within account X” is a POST /auth/select-account that the is_super_admin gate authorizes without a membership row (23_* §2.1).

Alternative (flagged, not chosen): embed a tenants[] array of scoped role-sets in one token and validate the chosen account_id per request (the JWT-claims guide lists both). *Tradeoff:* the array avoids the re-exchange round trip but ships every membership in one credential (larger blast radius on leak, and every downstream layer MUST re-validate the chosen tenant or lateral movement is possible — the anti-pattern the guide warns against). **Recommendation: token-exchange over the tenants-array**, for least standing exposure and smallest new surface. The final claim encoding + the exchange endpoint name are 21_*/this doc’s to bind together; 24_* §1.3 Option B is the same exchange reused for API keys (§4).

4. Credentials API — project-CLI API keys + device provisioning tokens

Two distinct machine credentials, both **scoped from a server-stored row, never self-asserted** (the resolvePublicKey/TrustTLSProxy trust boundary, 10_* §7; 24_* §1.1). Both consume 20_*’s api_keys/token shapes; neither is minted by an interactive password.

4.1 Project-CLI API keys (non-interactive CI/CD)

The `api_keys` shape exists **on paper only** (design-target migration `001_initial_schema.up.sql:57-71`, unimplemented in the shipped store — `10_* §1`) and gains the `account_id` + nullable `project_id` scope columns `20_*/24_* §1.1` own. New routes:

Method + path (new)	AuthZ	Purpose
POST <code>/admin/accounts/:accountId/api-keys</code>	requireSuperAdmin (or account admin, if <code>21_*</code> delegates)	mint a named, scoped key; cleartext returned ONCE , only <code>key_hash</code> stored (<code>001_initial_schema.up.sql:60</code> ; industry hash-only, <code>24_*</code> §Sources)
GET <code>/admin/accounts/:accountId/api-keys</code>	as above	list keys (redacted to <code>helixk_...<last4></code> , never cleartext — <code>24_*</code> §1.2)
DELETE <code>/admin/accounts/:accountId/api-keys/:keyId</code>	as above	revoke (set <code>revoked_at</code> ; immediate)
POST <code>/auth/token-exchange</code>	public (presents the key)	exchange the long-lived key for a short-lived scoped access token (<code>24_* §1.3 Option B</code> , the recommended CLI auth)

Recommendation: the key is an **exchange credential** (`24_* §1.3 Option B`) — the CLI calls `POST /auth/token-exchange` once per session and then uses the scoped bearer on the existing authenticated hot path (§2). *Tradeoff:* one extra round trip at session start + depends on the tenant claim in `Claims`; the alternative (direct-key Bearer on every request, Option A) is revocation-immediate but rides a long-lived secret on every call. Both are designed here; `21_*` picks whether the claim ships in milestone 1. This admin API also hosts, additively, the **per-account signing-key** surface (`POST/DELETE /admin/accounts/:accountId/signing-keys`) that `24_* §3 step 4` + `26_*` own — the verify key still comes only from server config/registry, never the request; only the *lookup key* gains an account dimension (`resolvePublicKey`, `handlers_artifact.go:274-283`). Flagged as a `26_*` dependency, not decided here.

4.2 Device provisioning tokens

Today device enrollment is operator-driven: `POST /devices/register` → `handleRegisterDevice` requires an operator/admin token (`server.go:192`) and mints a device-role token via `mintDeviceToken` (`sub = deviceId, handlers_device.go:157-159`) that carries **no account/project** (25_* §1.1). Two provisioning shapes (25_* §1.3):

Method + path	AuthZ	Model
<code>POST /devices/register</code> (existing <code>server.go:192</code> , extended)	<code>requireAccountAccess(operator)</code>	Option A — <code>mintDeviceToken</code> also stamps <code>account_id/project_id</code> from the operator's claim into the device token Option B — issue a short-lived, minimally-privileged bootstrap claim; the device exchanges it at first boot for a long-lived per-device (<code>account,project,deviceId</code>) token (AWS IoT fleet-provisioning-by-claim, 25_* §Sources)
<code>POST /admin/accounts/:accountId/enrollment-claims</code> + <code>POST /devices/claim</code> (new)	<code>mint: super-admin/account admin; claim: public</code> (presents the bootstrap claim)	

Recommendation: Option A now (enrollment is already operator-driven — the production delta is just “stamp the claim”), **Option B when self-service or fleet-scale enrollment is required** (they compose — B is A plus a claim-exchange front door). *Tradeoff:* A puts a long-lived token on-device (higher blast radius on leak) but needs no new enrollment protocol; B's on-device secret is per-device + rotatable but adds a claim lifecycle + a provisioning-hook to build. The device token's account claim is **server-minted** + **server-verified** — the device can never assert its own tenancy (25_* §1.2 trust boundary). Final token/enrollment shape = 20_*/21_*/25_*

5. Missing endpoints the as-is state requires

Beyond the admin/selection/credential routes above, the as-is audits flag concrete gaps the UI (23_*) and CLI (24_*) block on:

1. **GET /artifacts (list-artifacts) — the load-bearing gap.** The server exposes **no list-artifacts endpoint** today — only GET /artifacts/:artifactId metadata (server.go:198); the SPA documents this as a KNOWN GAP that leaves the release picker's artifact list empty (clients/ota-manager/src/hooks/useUploadArtifact.ts:4-7), and the CLI's helix-ota artifacts list depends on it (24_* §2, §4). **Add** GET /artifacts, account-scoped via the claim (§2) + optional ?project= narrowing, with the **existing** cursor+limit pagination convention (§6.3). AuthZ: requireAccountAccess(viewer).
2. **GET /auth/accounts + POST /auth/select-account** — §3 (account selection).
3. **GET /admin/accounts / GET /admin/users / GET /admin/accounts/:id/members** — §1 (super-admin console lists, 23_* §3.1).
4. **GET /admin/accounts/:id/api-keys** — §4.1 (key management UI + CLI).
5. **GET /projects becomes account-scoped** — it already exists (server.go:247) but returns the global project set; under §2 it returns only the active account's projects, which is what the SPA's real (non-mock) account/project switcher needs (23_* §2.2; use-projects.ts already anticipates a real GET /projects).
6. **Small wire additions (not new endpoints, flagged):** a notes/release-notes field on the release + pass-through in the UpdateAvailable offer (handlers_client.go:70-101) so the device wizard can show release notes (25_* §3) — an additive optional field (§6.1), not a route.

Out of scope here (owned elsewhere, flagged not silently dropped): the **artifact byte download/object-storage** surface — bytes are validated then discarded and StorageRef is a placeholder (handlers_artifact.go:184; 11_* §Honest-gaps 1) — is a 24_* §4 dependency; the device download-URL resolution (handlers_client.go:232-234) becomes per-account but the Storage seam + signed URLs are 24_*/26_*. This doc designs the *control* surface, not the byte-transfer surface.

6. Versioning, backward-compat, error contracts, pagination

6.1 Keep `/api/v1` working — additive-only, no `/api/v2`

Every new route above is **additive under the existing `/api/v1` group** (`server.go:189`) — new endpoints (`admin/*`, `auth/accounts`, `auth/select-account`, `auth/token-exchange`, `GET /artifacts`, `enrollment-claims`), new **optional** response fields (`account_id/project_id` on `Artifact/Release/Deployment/Device` wire shapes; a `release notes` field), and new **optional** query params (`?project=`). Per the versioning guidance (`$Sources`), “adding new endpoints, adding new properties to a response, and introducing optional parameters are non-breaking changes” — so **no `/api/v2` is required** for the multi-account surface.

The one potentially-breaking change is the newly-mandatory account scope on the existing OTA routes. Mitigation (composing `20_* $5` Option A default-account backfill): a **legacy token that carries no account claim resolves server-side to the seeded `__default__` account** — so a legacy single-tenant caller keeps working on the same paths, unchanged, during migration; the account is derived server-side, the URL never changes. **Recommendation:** keep the default-account fallback for a bounded transition window, then signal removal with a `Sunset` header on the legacy-unscoped behavior before dropping it (the versioning guidance’s deprecation pattern — `$Sources`), never a hard cutover. *Tradeoff:* the fallback keeps a “`NULL/__default__` account” path alive during migration (a small isolation caveat `20_* $5` accepts because the store is an in-memory MVP with negligible data); an operator with real multi-tenant prod data from day one drops the fallback and requires the claim immediately (`20_* $5` operator decision point).

6.2 Error contract — reuse the existing envelope + codes

The server already ships a stable error envelope `ErrorBody{code, message, details}` (`errors.go:32-34`) with helpers `respondError` (`errors.go:48`) / `respondValidation` (`errors.go:61`) over a closed code set (`errors.go:11-22`): `UNAUTHENTICATED`, `FORBIDDEN`, `NOT_FOUND`, `VALIDATION_FAILED`, `CONFLICT`, `UNSUPPORTED_MEDIA_TYPE`,

PAYLOAD_TOO_LARGE, RATE_LIMITED, SIGNATURE_INVALID, HASH_MISMATCH, VERSION_NOT_MONOTONIC, INTERNAL. The multi-account surface **reuses this envelope + codes — no new codes are required:**

- Invalid / expired / revoked API key or session, or missing account claim → UNAUTHENTICATED.
- Caller lacks the membership role for the route, or a super-admin-only route → FORBIDDEN.
- Per-account uniqueness violation (project/device-hardware/group name now unique **per-account**, 20_* §2.2) → the existing CONFLICT (as handleCreateDeployment already conflicts on an active target, 11_* §1).
- **Cross-tenant access — recommend NOT_FOUND, not FORBIDDEN.**
When account A requests a resource owned by account B, returning NOT_FOUND (rather than FORBIDDEN) avoids disclosing that the resource exists in another tenant — the anti-enumeration posture (Azure assume-breach + high-entropy tenant ids, §Sources).
Tradeoff: NOT_FOUND is slightly less diagnostic for a legitimately-confused caller; the isolation win dominates.
26_security_threat_model.md owns the final hard rule + its paired-mutation proof — flagged, not decided here.

6.3 Pagination — reuse the existing cursor+limit convention

Pagination **already exists** and is consistent: list endpoints take ?cursor= + ?limit= (default 50, validated to [1,200]) and return a NextCursor — see list-releases (handlers_release.go:106-130) and list-devices (handlers_device.go:107-138), backed by the Cursor/Limit fields on AuditFilter/DeviceFilter/ReleaseFilter (store.go:259-285). Every **new** list endpoint (accounts, users, members, api-keys, artifacts) **MUST** adopt the **same** cursor+limit+NextCursor shape, and the account/project scope is added as AccountID/ProjectID fields on those same *Filter structs (20_* §3.2) so the list paths gain scope with zero signature churn. Reusing the shipped convention (rather than inventing offset/page params) keeps the API self-consistent — a §11.4.50 determinism + consistency win.

Sources verified 2026-07-10

External research per §11.4.8 / §11.4.99 — REST multi-tenant API scoping (path vs token-claim), tenant-request mapping + request validation, admin/API-key credential design, and additive-versioning backward-compat. Findings corroborate the design: derive tenant from the signed token claim (not a request-supplied path/header) on the hot path, name the tenant explicitly only for cross-account admin operations, exchange tokens on account switch, and add tenancy as an additive `/api/v1` change (new endpoints + optional fields), not a new major version.

- **Microsoft Azure Architecture Center — “Map requests to tenants in a multitenant solution”** (ms.date 2024-11-24, page updated 2025-10-30): enumerates domain/subdomain, URL-path, query-string, custom-header, token-claim, API-key, and client-cert tenant identification; the **Request validation** section is load-bearing — “even though the request includes a domain name or other tenant identifier, it doesn’t mean you should automatically grant access” (assume-breach), and the API-keys note that short-lived claims-based tokens are “a more modern and secure approach” than long-lived keys. Grounds §2.0 (path is client-supplied → must be authorized), §2.1 (validate every request), §4 (API-key = record tenant per key, hash-lookup, expire/rotate). <https://learn.microsoft.com/en-us/azure/architecture/guide/multitenant/considerations/map-requests>
- **Multi-Tenant SaaS Architecture Hub — “JWT Claims for Tenant Scoping: Best Practices”**: top-level immutable `tenant_id` inside the signed payload read from one place, “if tenant scope is reconstructed from request metadata after verification, the signature protects nothing useful”; multi-tenant users → “issue one token per active tenant session” and switch context explicitly, or a tenants array validated per request; a flat global role list enables lateral movement. Grounds §2.0/§2.1 (claim, never self-asserted), §3 (token-exchange on selection/switch over a tenants-array). <https://www.multi-tenant-saas.com/auth-isolation-cross-tenant-access-control/tenant-aware-jwt-token-management/jwt-claims-for-tenant-scoping-best-practices/>
- **apifyflux — “Multi-Tenancy in REST API: Scalable, Secure Tenant Identification & API Designs”**: path-based (`api.example.com/tenant1/resources`) is “straightforward and easy to implement” but couples tenant to the URL; token-based “decoupling tenant data from the URL” is the “go-to approach for modern API designs” for security + scale; “a combination of these strategies is sometimes

employed to maximize both security and performance.” Grounds §2.0 (path-vs-token tradeoff + the hybrid recommendation). <https://www.apyflux.com/blogs/api-development/multi-tenancy-rest-api>

- **Speakeasy — “Versioning Best Practices in REST API Design”:**
“adding new endpoints, adding new properties to a response, and introducing optional parameters are non-breaking changes”;
“removing or renaming endpoints, removing required fields, and changing response structures are considered breaking changes”;
use a Sunset header + a transition window rather than an immediate cutover. Grounds §6.1 (additive tenancy needs no /api/v2; default-account fallback + Sunset deprecation). <https://www.speakeasy.com/api-design/versioning/>

No external source contradicts the proposed model. Negative finding (§11.4.99): none of the surveyed sources prescribes returning NOT_FOUND vs FORBIDDEN for cross-tenant denial — that recommendation (§6.2) follows from the Azure assume-breach + anti-enumeration principle applied to this API, and its hard rule is deferred to 26_*, not asserted as external consensus.

Honest boundary (§11.4.6)

- **Design proposal, not implemented.** No route, handler, middleware (requireSuperAdmin, requireAccountAccess), or wire field described here exists in the codebase yet. Every “as-is” fact about an existing route carries a first-hand file:line (server.go:183-251, handlers_auth.go, errors.go, handlers_release.go / handlers_device.go pagination, store.go:259-285); every “to-be” route is a proposal. Nothing here is a §11.4 completion claim.
- **Entity + claim + permission shapes are NOT decided here.** Account/User/Membership/Role and the api_keys scope columns are owned by 20_* (SSOT); the token-claim field names + the super-admin claim encoding + the permission-model shape (OQ-2) are 21_*. This doc fixes only route shapes, authZ gates, and the claim-derivation *direction*; where 20_*/21_* finalize a different shape, they win and this doc is reconciled to them.
- **Cross-cutting dependencies are flagged, not silently owned:** the tenant token claim (21_*), per-account signing-key registry + the cross-tenant NOT_FOUND rule + isolation proofs (26_*), real object storage + the artifact download surface (24_* §4), the device enrollment/provisioning model (25_*), and the default-account backfill/migration path (20_* §5). The API surface is designable and

reviewable now, but a live end-to-end “scoped upload → device downloads the exact bytes” cannot be proven until object storage lands — stated as fact, per 24_* §4 / 11_* §Honest-gaps 1.

- **Every open choice is a recommendation with tradeoffs for operator decision, never silently decided (§11.4.66):** token-claim tenancy on the hot path + path-based only for the cross-account admin API (§2.0); token-exchange over a tenants-array for account selection (§3); CLI key as an exchange credential — Option B (§4.1); operator-minted device token — Option A now, bootstrap-claim — Option B at scale (§4.2); additive `/api/v1`
 - default-account fallback + Sunset deprecation over a `/api/v2` cutover (§6.1); `NOT_FOUND` over `FORBIDDEN` for cross-tenant denial (§6.2, hard rule deferred to 26_*).
- **Scope of the audit behind the as-is facts:** route lines were read first-hand from non-test source (`server.go`, `handlers_auth.go`, `errors.go`, the two list handlers, `store.go` filters); the sibling docs’ `file:line` claims (token/claims, project ACL, device-token mint, `StorageRef` placeholder) are cited to 10_*/11_*/20_*/24_*/25_* and not independently re-read here. No test path (`*_test.go`) was audited.